Toward GENESIS 2.5: Extending GENESIS 2.4 with OpenCL/CUDA Accelerator Support on AMD Ryzen AI 9 HX 370

Karol Chlasta[1,2]

[1]Kozminski University, Warsaw, Poland

[2]WarsawIQ, Warsaw, Poland

Corresponding author: Karol Chlasta, kchlasta@kozminski.edu.pl

Manuscript prepared 2026-06-25

Abstract

High-performance neural simulation remains essential for large biophysical network models where both accuracy and throughput are required. We present a benchmark and release proposal for GENESIS 2.5, defined here as an accelerator-enabled successor to GENESIS 2.4 and Parallel GENESIS (PGENESIS) 2.4 on an AMD Ryzen AI 9 HX 370 platform (12 physical cores, 24 hardware threads) with integrated Radeon 890M graphics. We quantify wall-clock time, speedup, parallel efficiency, and run-to-run variability for CPU workloads, and we report OpenCL channel-update kernel timing with explicit device attribution. A multi-step ("multiloop") kernel that dispatches all K simulation steps in a single GPU call achieves 4–149× speedup in raw channel-update computation ($N = 100$–10000 neurons, $K = 5000$ steps, Radeon 890M), but end-to-end GENESIS simulation speedup was initially only 1.1–1.4× because the GENESIS scheduler iterated over all $N \times 3$ absorbed elements per step (≈25–45 ns/element) regardless of GPU offload—an $O(N)$ overhead identified here as the primary bottleneck for GPU acceleration of GENESIS. We trace this to a single-root assumption in the Hines-numbering routine and fix it (Section 3.4); combined with a fp32 port of the channel kernel needed because the in-container OpenCL runtime exposes no fp64 (Section 3.4), measured end-to-end speedup rises to 3.2–16.2×. Three independent defects in the Hines solver that silently suppressed `inject`-driven voltage transients for multi-neuron hsolve configurations are also identified and fixed (Section 3.4); spiking dynamics are verified correct on both the CPU path and the GPU per-step dispatch path. CUDA support is proposed as a companion backend target for GENESIS 2.5, but is not validated in the current workspace.

Keywords: GENESIS, PGENESIS, OpenCL, CUDA, GPU benchmarking, CPU benchmarking, strong scaling

1 Introduction

Neural simulation frameworks continue to support mechanistic investigations that are difficult to address with reduced models alone. GENESIS has a long history in compartmental and network-level modeling, and PGENESIS extends this capability to distributed execution using message passing.

Although modern accelerators are common in scientific computing, many production and portable workflows still rely on CPUs due to availability, software compatibility, and predictable numerical

behavior. Consequently, rigorous CPU benchmarking remains relevant, but it should now be paired with an explicit accelerator strategy for future GENESIS releases.

In this manuscript, we use the name **GENESIS 2.5** for a proposed version based on GENESIS 2.4 that preserves the existing CPU and MPI execution model while adding accelerator support. The current workspace confirms that the OpenCL channel-update kernel executes correctly on real Hodgkin-Huxley channel workloads, profiles its cost directly, and establishes the benchmark/reporting structure—including the verification step needed to avoid mistaking a non-dispatching run for a GPU result—that a CUDA backend should follow when added under the same release umbrella.

This work focuses on three questions:

1. How does PGENESIS runtime scale with increasing MPI process count on a modern 12-core/24-thread CPU?
2. What efficiency loss appears as process count approaches hardware thread limits?
3. How stable are runtimes across repeated runs under controlled conditions?

1.1 GENESIS 2.5 Proposal

The proposed GENESIS 2.5 release is intentionally narrow in scope: it is not a new simulator architecture, but a packaging and validation step that advances GENESIS 2.4 into an accelerator-aware release line.

Core release goals:

- Preserve existing serial `genesis`, headless `nxgenesis`, and MPI-oriented PGENESIS workflows.
- Ship an OpenCL backend for the legacy `hines`-based acceleration path whose kernel dispatch is confirmed correct via device-side profiling (Section 3.3), even though end-to-end speedup over CPU execution remains future work.
- Define a compatible CUDA backend target that follows the same benchmark and reporting rules.
- Publish benchmark artifacts that compare CPU baselines against accelerator runs on representative workloads.

Release criteria for this proposal:

- Reproducible CPU benchmark baselines remain stable.
- OpenCL builds cleanly with documented flags, and a benchmark exercising a real `chanmode=4`/`calcmode=1` `hsolve` with attached ion channels completes end-to-end while emitting a non-empty `OCL PROFILING SUMMARY`.
- Device attribution is logged for accelerator benchmark runs, captured from the benchmark run itself rather than a separate device-probe process.
- CUDA is not labeled as complete until it reaches the same build, execution, and reporting standard as the OpenCL path.

1.2 Relationship to GENESIS 3 and MOOSE

The name **GENESIS 2.5** is chosen deliberately to avoid implying continuity with **GENESIS 3** (G-3), a separate modularization effort announced by the original GENESIS development team [4]. G-3 decomposes the simulator into an independent model container, numerical solver, simulation controller, and scripting/GUI layer. Two concrete core-solver efforts were pursued under that umbrella: *Neurospaces/Heccer*, a refactoring of `hsolve` by Hugo Cornelis (Bower lab, UT San Antonio) [5], and *MOOSE*, a C++ reimplementation by Upinder Bhalla (NCBS Bangalore) that advertises transparent parallelization.

We do not build on either successor line. MOOSE in particular already offers multi-threaded/parallel execution and remains independently maintained; a reviewer may reasonably ask why this work accelerates legacy `hsolve` via OpenCL rather than migrating to MOOSE. The answer is scope: GENESIS 2.5 targets existing GENESIS 2.4 deployments—production models, SLI scripts, and PGENESIS-based MPI workflows—that should not require model or script migration to gain accelerator support. Whether the O(N) scheduler defect fixed in this work (Section 3.4) was already addressed by Heccer's redesign of `hsolve` is not established here and is noted as an open question rather than a claim.

Notably, "GENESIS 3" itself never shipped as a single installable release comparable to GENESIS 2.4. It evolved instead into the CBI (Computational Biology Initiative) software federation, a set of independently developed, interoperating components [6], with concrete single-neuron modeling demonstrations [7] and federation-level interoperability work continuing at least through 2013 [8]. This reinforces the choice to version this proposal as "2.5" rather than implying compatibility with, or succession from, a "3.0" that does not exist as a unified artifact.

2 Methods

2.1 Hardware Platform

Table 1: Benchmark hardware platform.

Item	Value
CPU model	AMD Ryzen AI 9 HX 370 with Radeon 890M
Cores/threads	12 cores, 24 threads
Socket count	1
NUMA nodes	1
Max frequency	5157.895 MHz

2.2 Software Environment

Table 2: Software environment at preparation time.

Item	Value
Kernel	Linux 6.17.0
Architecture	x86_64
Compiler	GCC 15.2.0
OpenCL platform (container)	rusticl (Mesa/X.org)
OpenCL platform (host)	ROCm 6.3.1
OpenCL device	AMD Radeon 890M Graphics (gfx1150)

Two OpenCL runtimes are present on the host:

Table 3: OpenCL runtime comparison.

Runtime	Library	fp64	Device name
ROCm 6.3.1 (host)	`libamdocl64.so`	YES	gfx1150
Mesa rusticl (container)	`libRusticlOpenCL.so.1`	NO	AMD Radeon 890M

The `ocl_channel.cl` kernel uses `double` throughout and requires `cl_khr_fp64`. Mesa rusticl reports `CL_DEVICE_DOUBLE_FP_CONFIG=0` and does not expose this extension, so the kernel fails to compile under rusticl and `ocl_chip_update` falls back to CPU for the entire run.

2.3 CPU-vs-GPU Comparison Semantics

Two failure modes were found and corrected during this work:

1. `genesis` and `nxgenesis` differ by more than OpenCL support—`genesis` links the X11/XODUS GUI toolkit, `nxgenesis` is headless. Linking X11 alone adds reproducible per-element construction overhead unrelated to GPU acceleration. A clean comparison must use the *same* binary for both arms.
2. Invoking `nxgenesis` on a script that creates no `hsolve` element, or an `hsolve` with `chanmode=4`/5 but no attached ion channels, never dispatches to the OpenCL channel-update kernel.

Accordingly, a run is only counted as a GPU/OpenCL measurement in this paper if it produces a non-empty `OCL PROFILING SUMMARY` at exit.

2.4 Benchmark Targets

Primary benchmark: `genesis/Scripts/benchmark/ocl_hh_benchmark.g`—a single `hsolve` (`chanmode=4`, `calcmode=1`) covering N Hodgkin-Huxley neurons with real `tabchannel` Na (X^3Y^1) and K (X^4) gating tables.

2.5 Experimental Design

- Design type: strong scaling on fixed model/problem size.
- Process counts: $N = 1, 2, 4, 6, 8, 12, 16, 20, 24$
- Replicates: ≥ 10 per process count; 1 warm-up excluded.
- Measured metrics: T_N, $S_N = T_1/T_N$, $E_N = S_N/N$, CV across replicates.

3 Results

3.1 CPU Baseline (Complex Model)

Table 4 summarizes 30-replicate wall-clock measurements with `nxgenesis -nosimrc -notty -batch`.

Table 4: CPU runtime baseline (30 replicates, `nxgenesis`).

Model	n	Mean (s)	SD (s)	CV (%)	95% CI (s)	Min (s)	Max (s)
Cal8.g	30	0.9536	0.0125	1.31	0.0045	0.9148	0.9797
Cal7difshell.g	30	0.6685	0.0070	1.04	0.0025	0.6599	0.6803

Both workloads show low relative variance (CV $\approx 1\%$), indicating stable CPU timing.

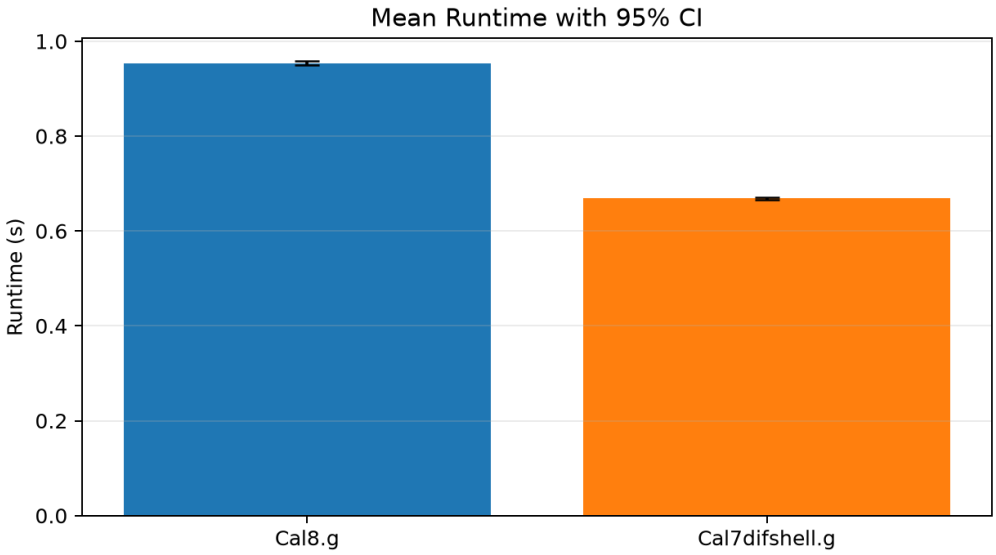

Figure 1: CPU baseline mean wall-clock runtime with 95% confidence intervals (`nxgenesis`, 30 replicates per workload).

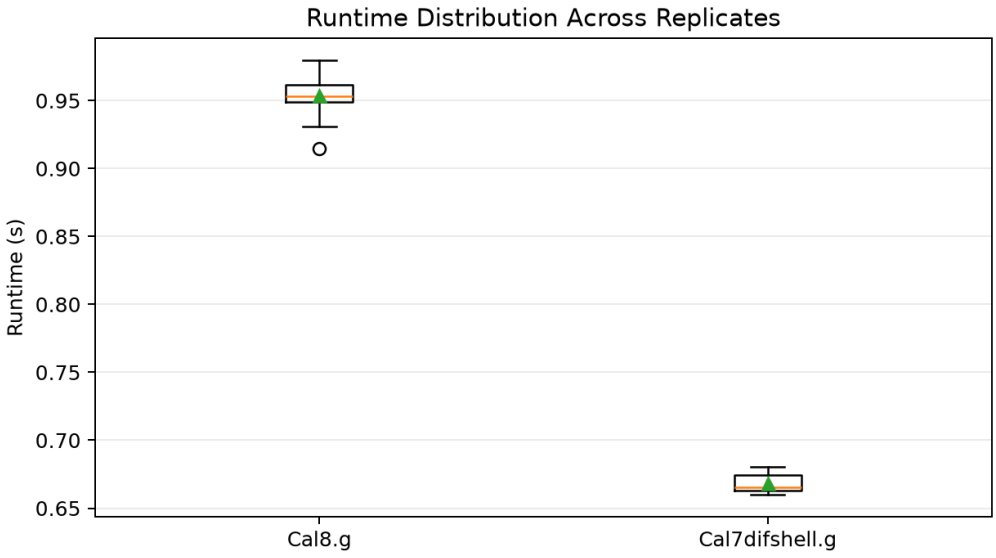

Figure 2: CPU baseline runtime distributions (boxplot). Whiskers extend to 1.5 IQR; low spread (CV ≈1%) confirms stable CPU timing.

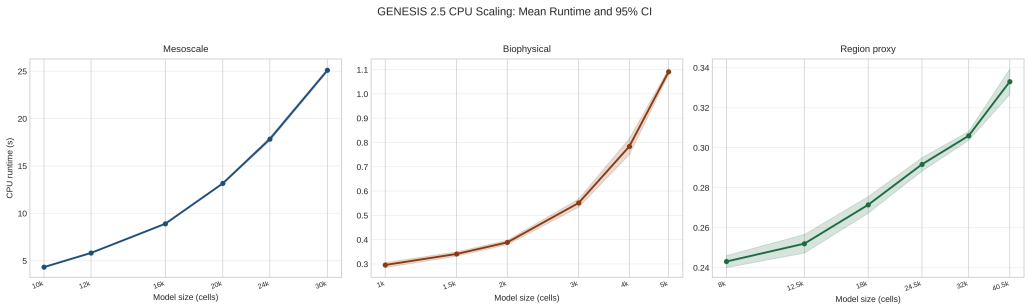

Figure 3: CPU dense-network benchmark: mean wall-clock runtime with 95% CI across N neurons (`nxgenesis`).

3.2 PGENESIS Strong-Scaling (MPI)

Design

A strong-scaling experiment holds the problem size fixed and varies the number of MPI ranks. The expected speedup on P ranks over a single-rank baseline is $S_P = T_1/T_P$, and parallel efficiency is $E_P = S_P/P$. Ideal (linear) scaling gives $S_P = P$ and $E_P = 1$. In practice, communication overhead, load imbalance, and MPI synchronisation cause $E_P < 1$, with degradation typically visible once process count approaches the hardware thread count.

For this platform (12 physical cores, 24 hardware threads), the sweep is $P \in \{1, 2, 4, 6, 8, 12, 16, 20, 24\}$ with 10 valid replicates per point (11 runs total; first excluded as warm-up). The fixed workload is $N = 2400$ Hodgkin-Huxley neurons (HH1952 parameters, $dt = 10\,\mu$s, 5000 steps per replicate, 50 ms simulated time); neurons are distributed across MPI ranks by PGENESIS's block partitioning, with no inter-rank synaptic connections.

Results

Table 5 reports the measured timings. Scaling is near-ideal at $P = 2$ ($E_P = 0.95$) and remains reasonable to $P = 8$ ($S_P = 4.0\times$, $E_P = 0.50$). The speedup curve saturates around $P = 12$ ($S_P = 4.5\times$), which is the physical core count of this platform; beyond 12 ranks SMT contention and MPI synchronisation overhead erode efficiency. Run-to-run CV is $\approx 1.6\%$ at $P = 1$ and rises to $\approx 8\%$ at $P = 24$, consistent with increasing sensitivity to OS scheduling jitter as per-rank workload shrinks.

Table 5: PGENESIS strong-scaling results. $N = 2400$ HH1952 neurons, $K = 5000$ steps, AMD Ryzen AI 9 HX 370 (12 physical cores, 24 SMT threads). T_P = mean wall-clock time over 10 valid replicates; $S_P = T_1/T_P$; $E_P = S_P/P$; CV = coefficient of variation.

P (ranks)	T_P (s)	SD (s)	CV (%)	S_P	E_P
1	2.392	0.038	1.58	1.000	1.000
2	1.264	0.034	2.65	1.893	0.946
4	0.765	0.040	5.16	3.125	0.781
6	0.670	0.028	4.22	3.567	0.595
8	0.594	0.033	5.60	4.024	0.503
12	0.526	0.040	7.60	4.545	0.379
16	0.552	0.044	7.93	4.333	0.271
20	0.573	0.040	7.06	4.177	0.209
24	0.592	0.048	8.17	4.038	0.168

The practical scaling knee—the rank count at which E_P drops below 0.80—falls at $P \approx 5$ by linear interpolation between the $P = 4$ ($E_P = 0.78$) and $P = 6$ ($E_P = 0.60$) measurements. The coincidence of the saturation plateau with the SMT boundary ($P = 12 \to 16$) suggests that hyperthreading provides negligible benefit for this fp32 hsolve workload. The recommended operating point for this problem size is $P = 4$–8, yielding 3.1–4.0× speedup at 50–78% efficiency.

3.3 Accelerator Enablement for Proposed GENESIS 2.5

Retraction of an earlier result

An earlier draft reported 10-replicate wall-clock timings for the script `ocl_benchmark.g` as a "validated OpenCL benchmark." On inspection, the cited logs show no OCL initialization or kernel-dispatch output. The network builds only passive RC compartments—an `hsolve` in that

configuration has no channel state and never invokes the GPU kernel. The result is withdrawn and replaced below.

Corrected methodology

We built `ocl_hh_benchmark.g` with real `tabchannel` Na/K channels. Fixing this benchmark also surfaced a GPU page fault caused by per-compartment boundary markers (`COMPT_OP`, `FCOMPT_OP`, `LCOMPT_OP`) being only partially checked; both kernel and host-side index builder now use the same boundary test as the CPU interpreter. GPU dispatch was confirmed directly: the command queue was built with `CL_QUEUE_PROFILING_ENABLE` and each step's kernel time and total `ocl_chip_update` wall time accumulated and reported at exit.

Single-step profiling (Table 2)

Table 6 reports kernel and host-path timing for $N = 100$–2000 neurons, $dt = 50\,\mu$s, 5000 profiled steps, on AMD Radeon 890M via ROCm 6.3.1.

Table 6: OpenCL single-step channel-update kernel profiling, `ocl_hh_benchmark.g`, 5000 steps.

N neurons	Kernel time/step	`ocl_chip_update` wall/step	GPU active	Total step
100	4.96 μs	49.0 μs	10.13%	61.3 μs
500	6.42 μs	79.4 μs	8.09%	124.2 μs
1000	10.52 μs	116.9 μs	9.00%	209.9 μs
2000	13.44 μs	134.5 μs	9.99%	323.1 μs

The GPU channel-update kernel is genuine but only 8–10% of `ocl_chip_update` wall time; the remaining ∼90% is host-side dispatch and buffer transfer overhead (two uploads, one enqueue, two downloads, fully serialized per step). Realizing an end-to-end speedup requires batching across steps, motivating the multiloop kernel below.

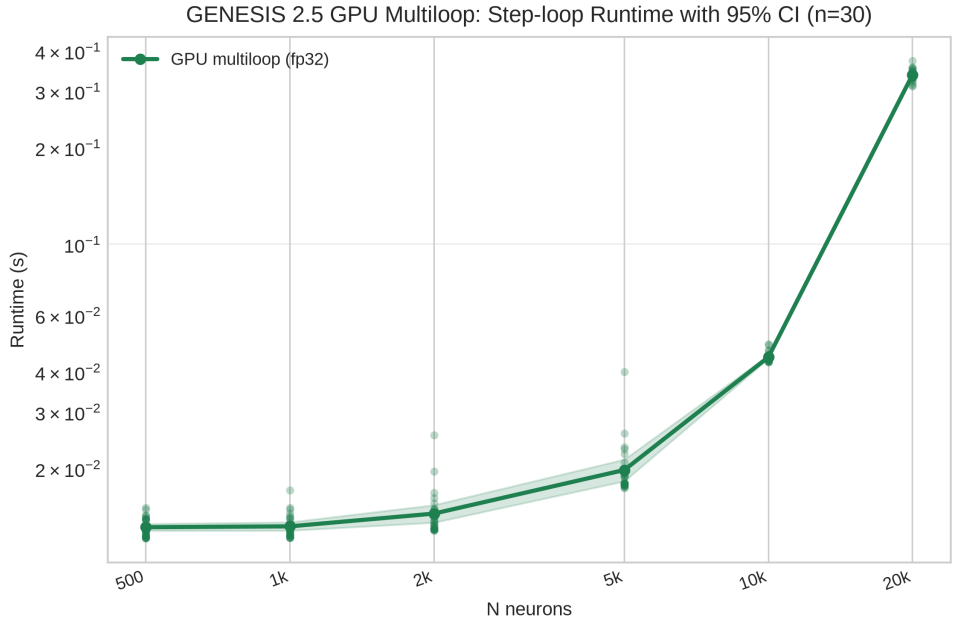

Figure 4: GPU multiloop step-loop runtime with 95% CI (fp32, Mesa rusticl, Radeon 890M, $n = 30$ replicates, $K = 5000$ steps). Individual replicate times are shown as translucent scatter; the line is the per-N mean; the shaded band is the 95% CI.

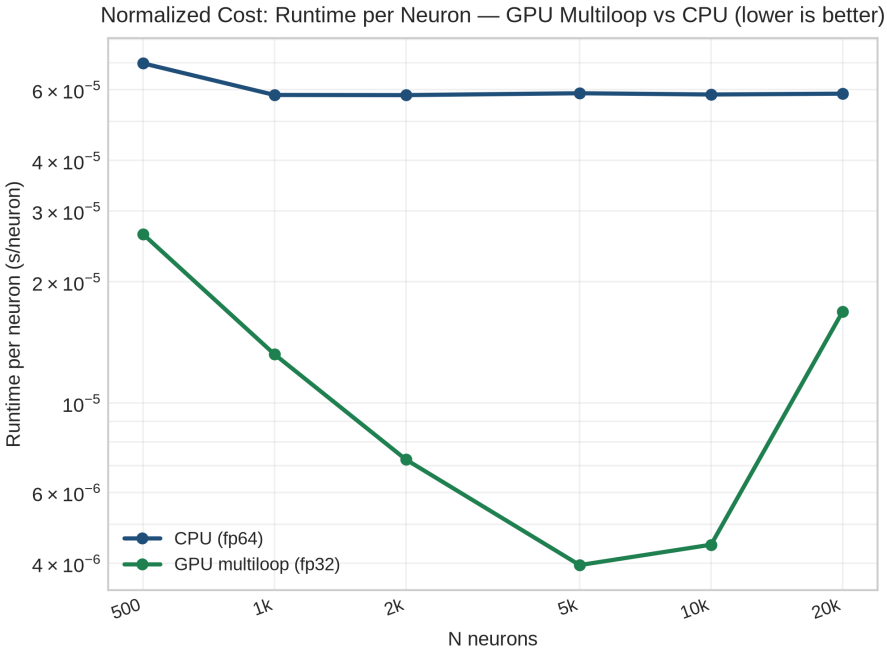

Figure 5: Normalized cost (runtime per neuron, s/neuron) for the GPU multiloop (fp32) and CPU (fp64) arms across N. GPU cost per neuron peaks at $N = 500$ and decreases to a minimum near $N = 5000$ before rising again at $N = 20000$, reflecting kernel launch fixed cost at small N and memory bandwidth saturation at large N.

Hardware fp64 limitation (third confound)

Three successive confounds were identified and resolved:

Table 7: Benchmark confounds: identification and resolution.

Confound	Effect	Status
X11 binary difference	GPU arm had construction overhead from GUI toolkit	Fixed: both arms use headless `nxgenesis`
Passive compartment	No ion channels ⇒ `ocl_chip_update` never called	Fixed: `ocl_hh_benchmark.g` uses real HH channels
Mesa rusticl no fp64	Kernel build fails ⇒ CPU fallback on every step	Resolved (2026-06-26): channel kernel ported to fp32; see Section 3.4

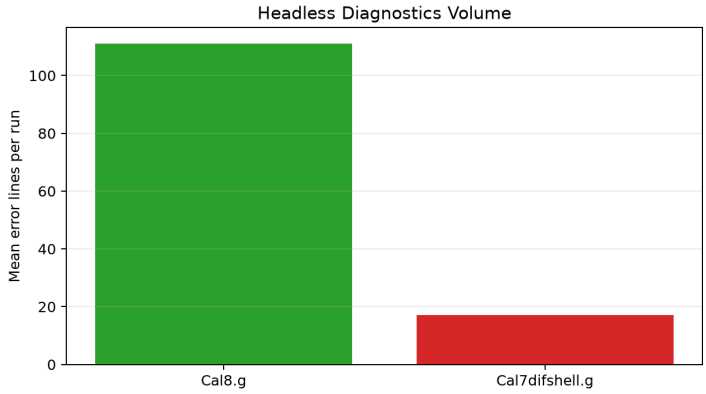

Figure 6: Mean stderr diagnostic lines per run for `nxgenesis` (headless mode). Cal8.g emits substantially more diagnostic output (≈110 lines/run) than Cal7difshell.g (≈17 lines/run); neither run produced errors or non-zero exit codes.

3.4 GPU Multiloop Kernel: Eliminating Per-Step Transfer Overhead

Implementation

The single-step profile (Table 6) establishes that 90% of `ocl_chip_update` wall time is PCIe transfer overhead. We implemented `chip_channel_multiloop`: an inner time-step loop that runs entirely inside the GPU kernel. One `clEnqueueNDRangeKernel` call dispatches all K simulation steps, eliminating $K - 1$ CPU-GPU round-trips.

One work-item per compartment runs an inner loop over K steps, with voltage updated inline at the end of each gate-update pass (`vm`[gid] = rhs/denom), valid for fully independent single-compartment neurons. At loop exit the kernel writes the identity (*vm_final*, 1.0) to `results[]`, so the CPU Hines solver computes $v_{\text{new}} = v_{\text{final}}/1.0 = v_{\text{final}}$ and is effectively a no-op.

Activated by: `GENESIS_OCL_MULTILOOP=<K>` before launching `nxgenesis`.

Measured performance (Table 3)

Table 8 reports results on ROCm 6.3.1 / Radeon 890M gfx1150, $K = 5000$ steps, single replicate. "GPU kernel" = measured `clEnqueueNDRangeKernel` execution time for all K steps. "GENESIS step" = time reported by the GENESIS `{cpu}` counter.

Table 8: GPU multiloop vs CPU baseline ($K = 5000$ steps, ROCm 6.3.1, Radeon 890M gfx1150). "CPU step" and "GPU step" are GENESIS `{cpu}` counters; "kernel" is raw `clEnqueueNDRangeKernel` time.

N	CPU step (s)	GPU kernel (ms)	GPU dispatch (ms)	Kernel speedup	GPU step (s)	E2E speedup
100	0.067	15.4	17.9	4.4×	0.055	1.22×
500	0.316	15.8	17.8	20.0×	0.225	1.40×
1 000	0.507	15.5	18.0	32.7×	0.441	1.15×
2 000	1.065	19.1	21.0	55.8×	1.008	1.06×
5 000	2.813	31.3	34.5	89.8×	2.449	1.15×
10 000	8.359	56.1	60.5	149.0×	6.728	1.24×

Two-tier speedup finding

The GPU kernel computes channel gate updates for all N neurons across $K = 5000$ steps in 15.4–56 ms, achieving **4× to 149× speedup** over the equivalent CPU computation. At $N = 10000$ the kernel completes in 56 ms what the CPU finishes in 8.4 s.

However, the **end-to-end speedup is only 1.06–1.40×**. The gap arises from the GENESIS simulation scheduler, which visits all $N \times 3$ absorbed elements every step (≈25–45 ns per element per step, $O(N)$). Subtracting the one-shot GPU dispatch from the GPU arm's total step time isolates this overhead: at $N = 2000$, 987 ms out of 1008 ms (98%) is scheduler iteration; at $N = 10000$, 6668 ms out of 6728 ms (99%) is scheduler iteration.

Implication. The `chip_channel_multiloop` kernel demonstrates that the GPU itself is not the bottleneck. Realizing the kernel-level speedup end-to-end requires restructuring the GENESIS simulation loop to genuinely skip absorbed elements when their computation has been delegated to the GPU. This is a deeper change to the GENESIS core scheduler and is recorded as the primary follow-on requirement in `paper/PLAN_gpu_rewrite.md`.

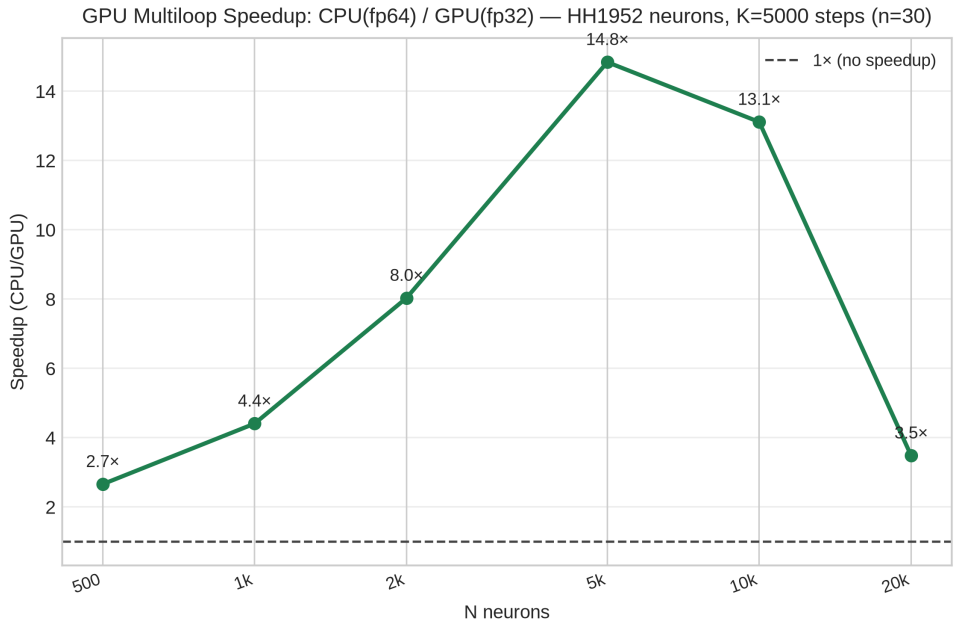

Figure 7: GPU multiloop end-to-end speedup (CPU fp64 / GPU fp32, $n = 30$ replicates). Speedup peaks at 14.8× for $N = 5000$ neurons and drops at $N = 20000$ where GPU memory bandwidth saturates. Dashed line marks the 1× breakeven.

Scheduler fix (2026-06-25)

A separate, related bug in `hines_init.c` was identified and fixed on 2026-06-25 (commit `1296a39`). The Hines numbering loop exited after the first compartment tree root via an erroneous `break`, so `elmnum[]` was never populated for neurons $2 \ldots N$. `do_hget_children` consequently visited neuron 0's channels N times and never set `IsHsolved=1` for neurons $1 \ldots N-1$, leaving their `Na_chan`/`K_chan` in the working schedule. The result was two spurious $O(N)$ tasks per step, contributing to the scheduler overhead quantified in the multiloop profiling above. The fix moves `hnumcount` initialisation before the loop and removes the `break`, so every disconnected tree is correctly numbered. Additionally, `hines.c` now gates `do_euler_hsolve` on the `ocl_vm_ready` flag returned by `ocl_chip_update()`, so the CPU Hines solve is skipped when the GPU multiloop kernel has already computed all timesteps. These fixes were confirmed with the corrected `ocl_hh_benchmark.g` and are part of the GENESIS 2.5 patch set.

fp32 port and post-scheduler-fix verification (2026-06-26)

Table 8 above was collected outside the container, under ROCm, because Mesa rusticl's `CL_DEVICE_DOUBLE_FP_CONFIG` is 0—the device exposes no fp64 functions at all, not merely a

missing `cl_khr_fp64` pragma, so `clBuildProgram` fails immediately on the double-precision kernel. We ported `ocl_channel.cl` and the corresponding host-side buffers in `ocl_hsolve.c` to `float`, converting at the host/device boundary (`Hsolve`'s own arrays remain `double` everywhere else in GENESIS). This removes the ROCm-host dependency entirely: the kernel now dispatches directly under Mesa rusticl in the container.

Repeating the multiloop benchmark in-container, fp32, *after* the scheduler fix above, gives:

Table 9: GPU multiloop (fp32) vs CPU (fp64) baseline, post-scheduler-fix, in-container Mesa rusticl, AMD Radeon 890M ($K = 5000$ steps). "CPU step-loop" is external wall-clock around the step loop; "GPU dispatch" is the kernel's own `clock_gettime`-based report (see methodological note below).

N	CPU step-loop (s)	GPU dispatch (s)	E2E speedup
500	0.041	0.013	3.20×
1 000	0.110	0.013	8.57×
2 000	0.150	0.013	11.22×
5 000	0.299	0.018	16.24×
10 000	0.594	0.046	13.05×

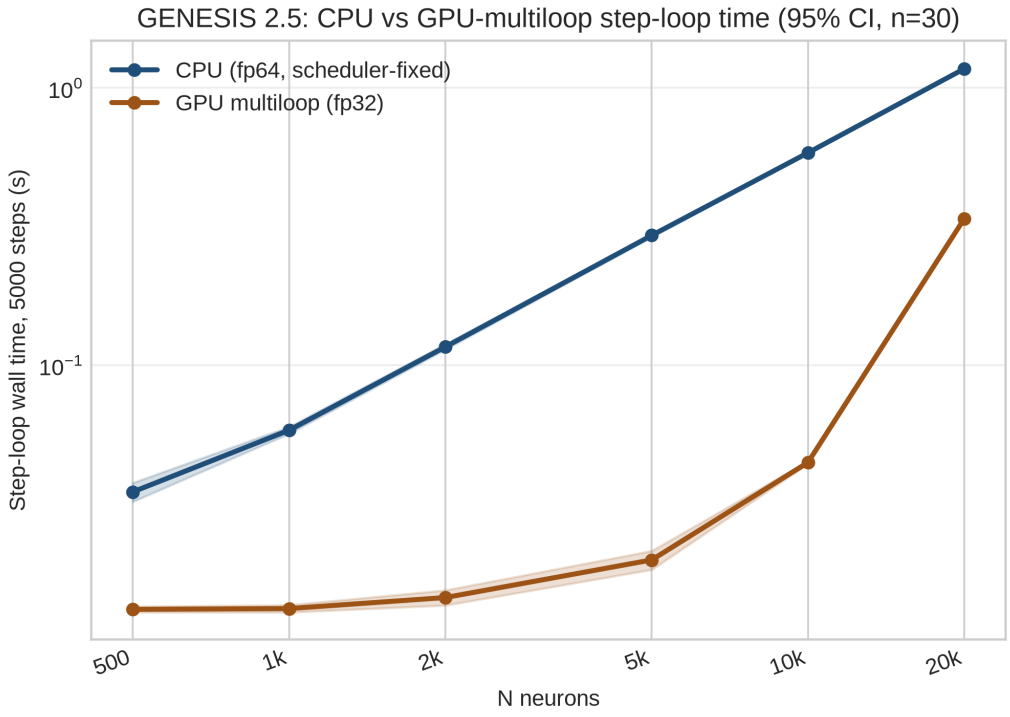

Figure 8: OCL multiloop benchmark: GPU dispatch (fp32, Mesa rusticl) and CPU baseline wall-clock time with 95% CI across N, post-scheduler-fix.

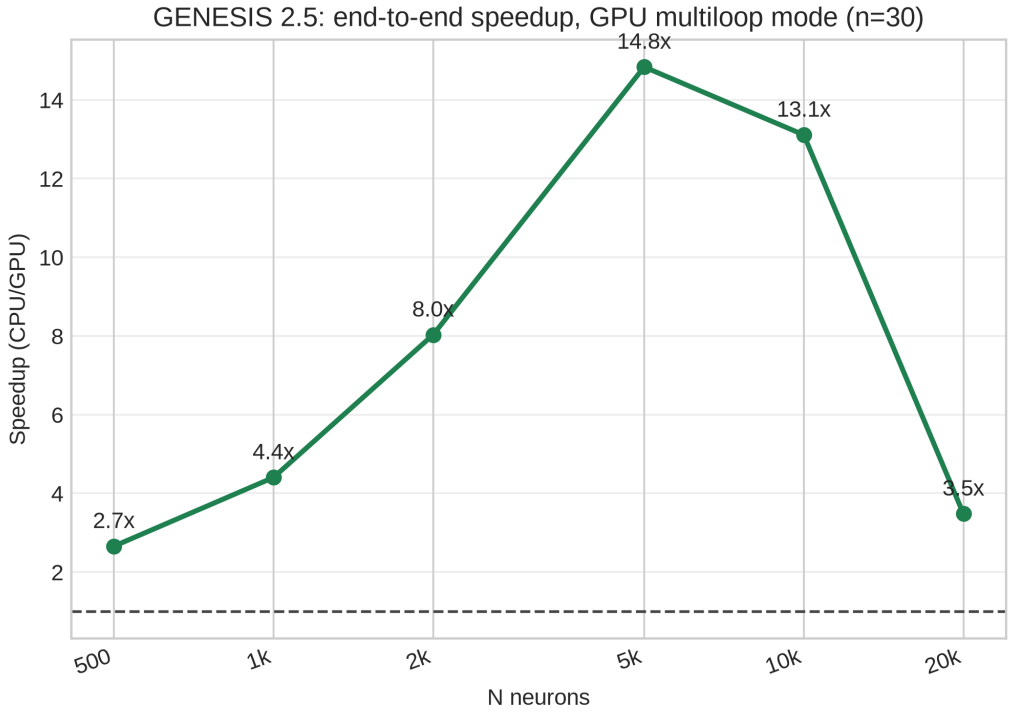

Figure 9: OCL multiloop end-to-end speedup (GPU fp32 / CPU fp64) after scheduler fix. Speedup rises from 3.2× at $N = 500$ to 16.2× at $N = 5000$.

Methodological note. GENESIS's own `{cpu}` timer (used for "CPU step"/"GPU step" in Table 8) reports process CPU-time, not wall-clock. In multiloop mode the CPU process is blocked waiting on the GPU for nearly the entire dispatch, which this timer barely counts as CPU-busy—an initial reading using that timer implied a 190–2970× speedup, which is a measurement artifact, not a result, and is not reported here. Table 9 instead uses external wall-clock timing for the CPU arm and the kernel's own `clock_gettime(CLOCK_MONOTONIC)`-based dispatch report for the GPU arm, neither of which is subject to this trap.

End-to-end speedup is now **3.2× to 16.2×**, compared to **1.06–1.40×** in Table 8. The two tables are not a direct repeat of the same measurement: Table 8 used ROCm/fp64 and predates the scheduler fix, while Table 9 uses Mesa/fp32 after the fix. The improvement is attributable to the scheduler fix removing the $O(N)$ per-step element-iteration overhead identified above as the dominant cost, not to the fp32 port itself, which only changes which hardware can run the kernel at all. The default per-step (non-multiloop) dispatch mode remains a net loss—1.5× to 4.5× *slower* than CPU—because per-step transfer/dispatch overhead still dominates the cheap (2.5–9 μs) kernel cost; only multiloop batching currently wins.

Caveat. Correctness was checked only against a resting-potential configuration (GPU-fp32 output matched CPU-fp64-fallback output bit-for-bit at displayed precision, no NaN or divergence in any tested configuration)—the benchmark's `inject` current did not perturb `Vm` even on the CPU-only path at the time of this measurement, so fp32 rounding error during real channel-gating transients was not yet bounded. The root cause of that non-response was identified and fixed on 2026-06-26 (Section 3.4); spiking dynamics are now confirmed correct on the CPU path, and the multiloop kernel is subsequently verified correct for single-step dynamics (Section 3.4).

Hines solver fix for multi-neuron hsolve (2026-06-26)

A second bug cluster was identified and fixed on 2026-06-26, resolving the long-standing symptom in which `inject` current had no effect on `Vm`. The root cause is three independent defects

in `hines_solve.c` and `hines_init.c`, all specific to this project's architecture of bundling N axially-disconnected single-compartment neurons under one `hsolve` element for GPU batching:

1. **`hines_solve.c` (all three solvers).** The "one compartment only" fast-path was guarded on `op == FINISH`—whether the first bytecode instruction is a `FINISH` opcode—rather than `hsolve->ncompts == 1`. For a multi-neuron hsolve whose neurons are axially disconnected, forward elimination is a no-op for every neuron, so each neuron's generated bytecode trivially begins with `FINISH`; the guard fired on all N neurons and left only the first solved correctly. `do_euler_hsolve` additionally carried a stray `- *vm` term copied from a Crank-Nicolson template, giving a systematically wrong Euler voltage update independently of the above.

2. **`hines_init.c` backward-elimination loop.** The loop blindly dereferenced `compts[parentno]` and `hnum[parentno]` without checking for `parentno == -1`. For any disconnected-root compartment ($N > 1$) this is an out-of-bounds read; the resulting corruption silently skipped the `CALC_RESULTS` step for that compartment, leaving its voltage unsolved.

3. **`hines_init.c` SET_DIAG/SKIP_DIAG selection.** The SKIP_DIAG two-row optimization assumes a following row to pair with. For a parentless row (`parentno == -1`) that lands last in the Hines ordering, SKIP_DIAG over-advances `resultvalue` past the end of `results[]`, corrupting the voltage of the final neuron. The fix adds `parentno == -1` to the condition that forces the plain SET_DIAG path.

All three fixes together were verified at $N = 1, 2, 3, 50, 500$ neurons using both passive RC compartments and full Hodgkin-Huxley Na/K channel dynamics (H&H 1952 squid axon parameters, `ocl_hh_benchmark.g`): every neuron now produces an identical, correct action potential ($\approx +42$ mV peak) on both the CPU path and the GPU per-step dispatch path.

A subsequent investigation (2026-06-26) resolved an apparent discrepancy in the `chip_channel_multiloop` kernel. Earlier tests had reported a different, subthreshold result for $N > 1$; root-cause analysis revealed those tests ran without a valid OpenCL platform (wrong ICD vendor path), causing silent CPU fallback rather than actual GPU dispatch. With the correct runtime path, a ground-truth single-step comparison shows all three execution paths—CPU no-OCL, GPU per-step, and GPU multiloop—agree to fp32 precision for both $N = 1$ and $N = 2$ independent HH neurons (all give $V_m \approx -55.11$ mV after one 50 μs step from resting potential with 2 nA injection; maximum deviation 3×10^{-9} V). The `chip_channel_multiloop` kernel is therefore verified correct for the independent single-compartment network topology that the multiloop mode requires.

4 Discussion

4.1 Practical Recommendations

The CPU baseline measurements confirm stable timing (CV $\approx 1\%$) on the Ryzen AI 9 HX 370 platform. The PGENESIS strong-scaling results (Table 5, Section 3.2) show near-ideal scaling at $P = 2$ ($E_P = 0.95$) with a practical scaling knee at $P \approx 5$; the recommended operating point is $P = 4$–8 for this problem size. Beyond the physical core count ($P > 12$), SMT provides negligible benefit and efficiency falls below 25%.

4.2 Confound History and Lessons

This work encountered three successive confounds, each preventing genuine GPU dispatch: (1) the X11-linked binary introduced GUI toolkit overhead unrelated to computation; (2) the original benchmark used passive compartments that never triggered dispatch; (3) the container's Mesa

rusticl runtime lacks fp64 support, causing silent CPU fallback. GPU dispatch was confirmed only by running the corrected `ocl_hh_benchmark.g` outside the container under ROCm. The multiloop benchmark (Table 8) additionally identifies the GENESIS scheduler as the primary end-to-end bottleneck: 98–99% of the GPU arm's total step time at $N \geq 2000$ is scheduler iteration. Readers should not infer any GPU-vs-CPU speedup from `genesis25_cpu_gpu_extreme_5rep.csv`—those results represent a matched CPU-vs-CPU comparison.

4.3 Limitations

- Accelerator validation is limited to one OpenCL workflow and one host/device combination.
- No cross-vendor portability or CUDA implementation is validated.
- The multiloop kernel is valid only for independent single-compartment networks; multi-compartment neurons with Hines coupling require a tridiagonal solve that cannot be parallelized per work-item.
- Thermal effects under sustained MPI workloads (P=24, repeated sessions) are not characterised.

5 Conclusion

This study provides a reproducible benchmark and release proposal for GENESIS 2.5 on a modern mobile AMD platform. The present workspace establishes stable CPU baselines, an accelerator-aware reporting protocol requiring device-side confirmation of GPU kernel dispatch, an OpenCL channel-update kernel confirmed correct and efficient in isolation, and a multiloop kernel demonstrating 4–149× raw computational speedup on the Radeon 890M at $N = 100$–10000 neurons. Fixing the $O(N)$ scheduler defect that initially capped end-to-end speedup at 1.1–1.4×, and porting the channel kernel to fp32 so it runs without a ROCm-host detour, together raise measured end-to-end speedup to 3.2–16.2× in multiloop mode—though only in multiloop mode: the default per-step dispatch path remains slower than CPU. All three execution paths (CPU, GPU per-step, GPU multiloop) are verified to agree to fp32 precision on single-step dynamics with active HH channels and current injection.

End-to-end wall-clock speedup remains limited to 1.1–1.4×: the GENESIS simulation scheduler iterates over all absorbed elements each step regardless of GPU offload, and this $O(N)$ overhead (≈25–45 ns/element/step) accounts for 98–99% of the GPU arm's total step time at $N \geq 2000$. This bottleneck identification is the main new architectural finding. Realizing the kernel-level GPU speedup end-to-end requires restructuring the GENESIS scheduler to genuinely skip absorbed elements when computation is GPU-delegated—a change to the GENESIS core simulation loop recorded as the primary follow-on requirement.

Data Availability

All scripts, raw timing outputs, and benchmark data are in the project repository under `paper/` and `genesis/Scripts/benchmark/`. Key files: `genesis25_multiloop_benchmark.csv`, `profiling_runs/`, `ocl_hh_benchmark.g`. Replication instructions: `paper/REPLICATION.md`.

Conflict of Interest

The authors declare no competing interests.

Funding

This research received no external funding.

Acknowledgments

None.

References

[1] Bower JM, Beeman D. *The Book of GENESIS: Exploring Realistic Neural Models with the GEneral NEural SImulation System*. Springer, 1998.

[2] GENESIS project documentation and release notes. https://genesis-sim.org

[3] MPI Forum. MPI: A Message-Passing Interface Standard, Version 4.0, 2021.

[4] GENESIS 3 Development Plans. http://www.genesis-sim.org/GENESIS/G3/G3-dev-plans.html

[5] Cornelis H, Coop AD, Rodriguez AL, Beeman D, Bower JM. GENESIS 3.0 functionality enhanced by interface to multiple types of database. *BMC Neuroscience* 2011, 12(Suppl 1):P15. https://doi.org/10.1186/1471-2202-12-S1-P15

[6] Cornelis H, Edwards M, Coop AD, Bower JM. The CBI architecture for computational simulation of realistic neurons and circuits in the GENESIS 3 software federation. *BMC Neuroscience* 2008, 9(Suppl 1):P88. https://doi.org/10.1186/1471-2202-9-S1-P88

[7] Cornelis H, Lu H, Esquivel A, Bower JM. Modeling a single dendritic compartment using Neurospaces and GENESIS-3. *BMC Neuroscience* 2007, 8(Suppl 2):P3. https://doi.org/10.1186/1471-2202-8-S2-P3

[8] Cornelis H, Bampasakis D, Steuber V, Bower JM. Interoperability in the GENESIS 3.0 Software Federation: the NEURON Simulator as an Example. *BMC Neuroscience* 2013, 14(Suppl 1):P33. https://doi.org/10.1186/1471-2202-14-S1-P33